

0→1 Implementation Brief: Consumer-Grade Installer for Containerized Apps

Working Title

CrateRun — placeholder name.

Executive Summary

We want to build a developer tool that turns a containerized application, especially a Docker Compose app, into a normal consumer desktop installer.

The end user experience must be as simple as installing Chrome:

1. Download installer.
2. Double-click.
3. Click install.
4. App opens.

The user should never need to know Docker, Compose, WSL, ports, volumes, images, registries, or terminals exist. Those are implementation details hidden behind a native installer, a desktop app shell, and a local supervisor.

The core product promise is:

Turn a containerized app into a normal desktop app installer.

The 0→1 implementation should be narrow, opinionated, and boring. The goal is not to support every possible Compose file. The goal is to prove a single vertical slice where a developer packages a simple full-stack app and a mainstream Windows user can install and run it without technical knowledge.

North Star

A non-technical user should be able to install and run an app generated by this system without reading documentation.

The generated app should feel like a normal desktop application, not a Docker wrapper.

User-facing install flow

Install MyApp?

[Install]

Then:

Installing MyApp...

Then:

MyApp is ready.

[Open]

That is the bar.

No runtime selection.

No Docker Desktop install instructions.

No exposed ports.

No terminal output.

No Compose terminology.

No "localhost:3000."

0→1 Scope

Primary target

For the first true vertical slice, target:

OS: Windows 11 x64

Runtime: managed WSL2 distro + containerd/nerdctl or equivalent

App type: local full-stack web app

Shell: Tauri desktop shell

Distribution: single-file installer or installer + embedded app bundle

Windows should be first because it will expose the worst consumer-install problems early: permissions, WSL, virtualization, Defender warnings, code signing, reboots, path weirdness, and uninstall cleanup.

If we make this work cleanly on Windows, macOS and Linux become follow-on implementation targets instead of open-ended unknowns.

Supported app shape for MVP

The MVP should support apps shaped like this:

```
frontend service
backend/api service
optional worker service
optional database service
persistent app data volume
local web UI shown in desktop window
```

Example Compose app:

```
services:
  web:
    image: myapp-web
    ports:
      - "3000"

  api:
    image: myapp-api
    ports:
      - "8080"
    environment:
      DATABASE_URL: postgres://postgres:postgres@db:5432/app

  db:
    image: postgres:16
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

Not in MVP

Do not support these in the 0→1 release:

```
arbitrary host mounts
privileged containers
host networking
```

```
GPU passthrough
Kubernetes
Swarm
multi-user installs
LAN/server mode
Docker socket mounts
complex VPN/network requirements
auto-updates in background
registry-pull-only installs
macOS and Linux consumer flows
```

These are not impossible. They are just distractions from proving the core consumer experience.

Product Model

There are two audiences:

1. **Developer / publisher** — uses the CLI to package an app.
2. **End user / consumer** — runs the generated installer and uses the app.

The developer may understand Docker. The consumer should not have to.

Developer command target

The ideal developer flow:

```
craterun init
craterun doctor
craterun package --consumer --target windows-x64
```

Output:

```
dist/
  MyAppSetup.exe
```

End-user target

The user downloads and runs:

```
MyAppSetup.exe
```

They see:

```
Install MyApp?  
[Install]
```

Then the app installs, prepares its private runtime, starts its services, and opens in a native desktop window.

High-Level Architecture

Developer Machine

CrateRun CLI

- reads docker-compose.yml
- reads craterun.yml
- builds images
- locks image refs
- exports OCI archives
- builds app bundle
- builds native installer

MyAppSetup.exe

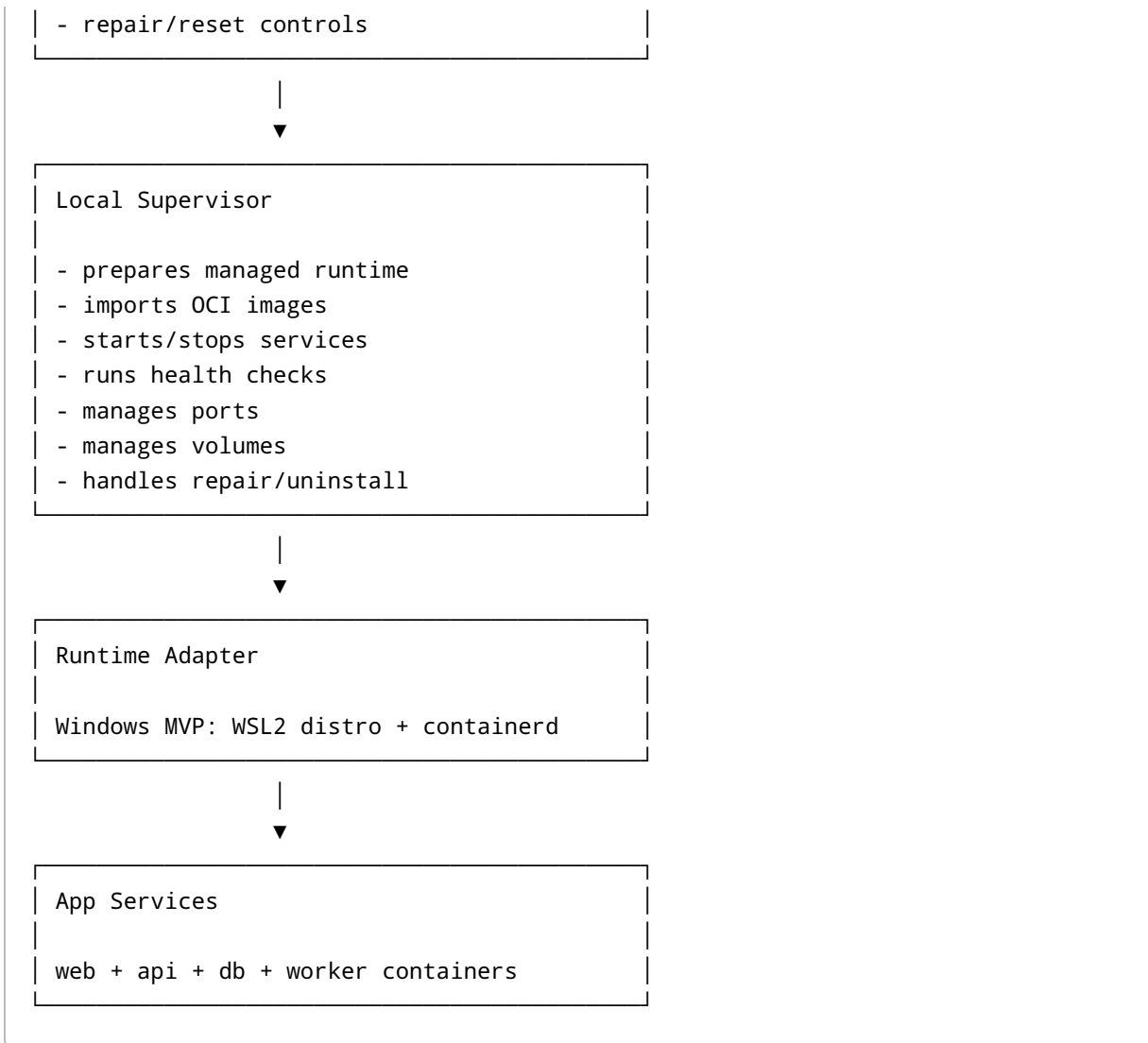
End-User Machine

Native Installer

- copies desktop shell
- copies supervisor
- copies runtime assets
- copies app bundle
- registers uninstall hooks

Desktop Shell

- installer progress UI
- app loading screen
- webview window
- diagnostics screen



Major Components

1. Developer CLI

The CLI is responsible for doing the heavy lifting on the developer machine.

Responsibilities

```
read craterun.yml
read docker-compose.yml
validate supported Compose subset
build local images
pin image digests
```

```
export images as OCI archives
compress app bundle
generate runtime manifest
generate locked compose plan
build installer payload
run consumer-readiness certification
```

Commands for MVP

```
craterun init
craterun doctor
craterun build
craterun package --consumer --target windows-x64
craterun run-local
craterun certify
```

CLI language recommendation

Use **Rust** or **Go**.

Rust is preferred if the supervisor will also be Rust, because shared manifest parsing, runtime code, and filesystem utilities can be reused.

Go is acceptable if speed of implementation is more important than long-term binary/library cohesion.

Recommended 0→1 choice:

```
Rust CLI + Rust supervisor + Tauri shell
```

This gives us:

```
small native binaries
strong process management
cross-platform future path
single language for low-level runtime operations
natural pairing with Tauri
```

2. App Manifest

Docker Compose describes infrastructure. It does not describe a consumer app.

We need a higher-level manifest, `craterun.yml`, that maps Compose services into desktop app concepts.

Example manifest

```
app:
  id: com.example.photobooth
  name: PhotoBooth Local
  version: 1.0.0
  icon: ./assets/icon.png

compose:
  file: ./docker-compose.yml
  projectName: photobooth

window:
  title: PhotoBooth Local
  width: 1200
  height: 800
  entry: /

routes:
  /:
    service: web
    port: 3000

  /api:
    service: api
    port: 8080

health:
  ready:
    service: api
    path: /health
    timeoutSeconds: 120

volumes:
  db-data:
    durability: persistent
    backup: true

lifecycle:
  onWindowOpen: startServices
  onWindowClose: stopServices

install:
```



```
mode: consumer
askQuestions: false
```

MVP manifest concepts

```
app metadata
icon
version
compose file path
primary window config
route mapping
health check
persistent volumes
lifecycle behavior
consumer install mode
```

Important design rule

The manifest should be app-oriented, not infrastructure-oriented.

Bad manifest concept:

```
containerNetworkBridgeMode
```

Good manifest concept:

```
routes
```

Bad manifest concept:

```
hostPort
```

Good manifest concept:

```
autoExposeFrontend
```

The platform should absorb infrastructure complexity where possible.

3. Bundle Format

The package output should include a self-contained app bundle.

Working extension:

```
.appcrate
```

The installer can either embed this bundle or download it as a signed payload. For the 0→1 version, prefer embedding it in the installer or placing it next to the installer as a required payload.

Bundle structure

```
myapp.appcrate
manifest.json
compose.lock.yml
images/
  web-linux-amd64.oci.tar.zst
  api-linux-amd64.oci.tar.zst
  db-linux-amd64.oci.tar.zst
assets/
  icon.png
shell/
  shell-config.json
sbom/
  bundle.sbom.json
signatures/
  bundle.sig
```

Manifest JSON output

`manifest.json` should be the compiled form of `craterun.yml`.

Example:

```
{
  "schemaVersion": "0.1",
  "app": {
    "id": "com.example.photobooth",
    "name": "PhotoBooth Local",
    "version": "1.0.0"
  },
  "runtime": {
    "target": "windows-x64",
```

```
    "architecture": "linux/amd64"
  },
  "services": [
    {
      "name": "web",
      "imageArchive": "images/web-linux-amd64.oci.tar.zst",
      "containerPort": 3000,
      "role": "frontend"
    },
    {
      "name": "api",
      "imageArchive": "images/api-linux-amd64.oci.tar.zst",
      "containerPort": 8080,
      "role": "backend"
    }
  ],
  "routes": [
    {
      "path": "/",
      "service": "web",
      "port": 3000
    },
    {
      "path": "/api",
      "service": "api",
      "port": 8080
    }
  ],
  "health": {
    "ready": {
      "service": "api",
      "path": "/health",
      "timeoutSeconds": 120
    }
  }
}
```

4. Compose Support

Do not try to support all of Compose at first.

Support a known-safe subset.

MVP-supported Compose features

```
services
images built locally
basic environment variables
named volumes
service dependencies
container ports
health checks where possible
restart policy mapping
```

MVP-blocked Compose features

Block these in consumer mode:

```
privileged: true
network_mode: host
pid: host
ipc: host
cap_add with dangerous capabilities
host root mounts
Docker socket mounts
arbitrary bind mounts outside app data directory
manual host ports
external networks
external volumes
custom low-level network drivers
```

Consumer certification should fail if unsafe features are present

Example output:

```
Consumer readiness failed

x api uses privileged mode
x worker mounts /var/run/docker.sock
x web exposes fixed host port 3000

Consumer installers cannot include these features.
Use --mode dev-pack if this is intentional.
```

This is essential. If developers can package arbitrary cursed Compose files, the user experience and security model are dead.

5. Native Installer

The installer should copy the app shell, supervisor, runtime assets, and app bundle to the user machine.

For 0→1, the installer can be simple. It does not need full enterprise deployment support yet.

Windows install location

Recommended per-user install:

```
%LOCALAPPDATA%\Programs\MyApp
```

Data location:

```
%LOCALAPPDATA%\CrateRun\Apps\com.example.myapp
```

Or app-specific branded path:

```
%LOCALAPPDATA%\MyApp\Data
```

For 0→1, app-specific paths may be easier for uninstall/debuggability.

Installed structure

```
%LOCALAPPDATA%\Programs\MyApp
  MyApp.exe
  myapp-supervisor.exe
  runtime-assets
  bundle
    myapp.appcrate
  uninstall.exe
```

Data/runtime structure:

```
%LOCALAPPDATA%\CrateRun\Apps\com.example.myapp
  runtime
  images
  volumes
  logs
```

```
config
state.json
```

Installer UI

The 0→1 installer should be minimal:

```
Install MyApp?

[Install]
```

Avoid advanced options during the happy path.

Installer responsibilities

```
copy application files
register Start Menu shortcut
register uninstall entry
launch app after install
```

The installer should not perform every runtime operation itself. It should install files and launch the desktop shell/supervisor. The supervisor should own app preparation because it can also repair/retry later.

6. Desktop Shell

The desktop shell is the user-facing application.

Recommended stack:

```
Tauri + React
```

Responsibilities

```
show install/setup progress
show loading screen
open web app in native window
communicate with supervisor
show diagnostics only when needed
provide repair/reset/export logs controls
```

Shell states

```
Preparing
Starting
Ready
Repairing
FailedRecoverable
FailedBlocked
```

User-facing startup flow

```
Opening MyApp...
```

Then if first run:

```
Setting things up...
This only happens the first time.
```

Then:

```
Starting MyApp...
```

Then the webview navigates to the local app proxy endpoint.

Important UX rule

The shell must never show raw container output unless the user explicitly opens diagnostics.

Bad:

```
Error: failed to solve: rpc error: code = Unknown desc = failed to copy
```

Good:

```
MyApp had trouble starting.

[Try Again] [Repair] [Show Details]
```

7. Local Supervisor

The supervisor is the most important component.

The shell is the face. The supervisor is the adult in the room.

Responsibilities

```
manage app lifecycle
prepare runtime
start runtime
import images
create app network
create volumes
start services
start reverse proxy
run health checks
track state
write logs
repair app
reset app data
uninstall app data/runtime
```

Supervisor interface

Expose a local control API over named pipe or loopback HTTP.

For Windows MVP, named pipe is preferable for local-only control, but loopback HTTP is faster to implement.

Initial API shape:

```
GET  /status
POST /prepare
POST /start
POST /stop
POST /restart
POST /repair
POST /reset-data
GET  /logs
GET  /diagnostics/export
```


Status response

```
{
  "appId": "com.example.photobooth",
  "state": "StartingServices",
  "message": "Starting PhotoBooth Local...",
  "technicalMessage": "waiting for api health check",
  "progress": 0.72,
  "services": [
    {
      "name": "db",
      "state": "running"
    },
    {
      "name": "api",
      "state": "starting"
    },
    {
      "name": "web",
      "state": "pending"
    }
  ]
}
```

Internal supervisor state machine

```
NotPrepared
CheckingSystem
PreparingRuntime
ImportingImages
CreatingNetwork
CreatingVolumes
StartingServices
StartingProxy
WaitingForHealth
Ready
Stopping
Stopped
Repairing
FailedRecoverable
FailedBlocked
```

Every state should have:

```
user-facing message
technical details
retry action
failure transition
log event
```

8. Runtime Strategy for Windows MVP

The cleanest consumer path is a private managed Linux runtime.

Recommended MVP path:

```
WSL2 managed distro + containerd + nerdctl-compatible orchestration
```

The generated app should not require Docker Desktop.

Why not Docker Desktop?

Docker Desktop is excellent for developers but is a bad consumer prerequisite. It introduces licensing, account, install, startup, update, and UX friction. The end user should not be installing Docker to run a normal app.

Runtime responsibilities

The runtime layer should provide:

```
import OCI image archive
start container
stop container
remove container
create volume
remove volume
create network
map ports to localhost
collect logs
inspect health/status
```

Adapter abstraction

Define a runtime adapter interface early even though the MVP only implements one adapter.

```
interface RuntimeAdapter {
  detect(): Promise<RuntimeDetectionResult>;
  prepare(): Promise<void>;
  importImage(archivePath: string): Promise<ImageRef>;
  startApp(plan: RuntimePlan): Promise<void>;
  stopApp(appId: string): Promise<void>;
  getStatus(appId: string): Promise<AppRuntimeStatus>;
  getLogs(appId: string, service?: string): AsyncIterable<LogLine>;
  removeApp(appId: string, options: RemoveOptions): Promise<void>;
}
```

In Rust this would become a trait.

Future adapters

```
WindowsWslContainerdAdapter
MacLimaContainerdAdapter
LinuxPodmanAdapter
LinuxDockerAdapter
ExistingDockerAdapter
ExistingPodmanAdapter
```

Do not bake WSL assumptions into the supervisor core.

9. Local Reverse Proxy

Every app should expose one private local endpoint to the desktop shell.

Instead of exposing frontend and backend separately, the supervisor starts a local reverse proxy.

Example:

```
127.0.0.1:<random-app-port>/      -> web:3000
127.0.0.1:<random-app-port>/api   -> api:8080
```

Benefits

```
one local port
no user-visible localhost details
no frontend CORS problems
clean app window URL
```

```
easier health checks  
easier future auth/session handling
```

MVP implementation

Implement proxy inside the Rust supervisor if practical.

Alternative: run a tiny bundled proxy binary.

Recommended:

```
Rust supervisor hosts reverse proxy directly
```

The shell asks supervisor for the app URL:

```
{  
  "state": "Ready",  
  "appUrl": "http://127.0.0.1:42137/"  
}
```

Then the shell loads that URL in the webview.

10. Ports

Never require the developer or end user to pick fixed host ports in consumer mode.

Rule

Container ports can be fixed. Host ports should be automatically assigned.

Bad consumer behavior:

```
Port 3000 is already in use.
```

Correct behavior:

```
Supervisor selects an available high port automatically.
```

The user should not see any of this.

Port policy

```
bind address: 127.0.0.1 only  
host port: randomly allocated high port  
route access: through local reverse proxy
```

11. Data Model

Persistent app data should be isolated and easy to remove.

App data root

```
%LOCALAPPDATA%\CrateRun\Apps\<appId>
```

Structure

```
state.json  
logs  
runtime  
volumes  
backups  
config
```

Volume durability model

The manifest should classify volumes:

```
volumes:  
  db-data:  
    durability: persistent  
    backup: true  
  
  cache-data:  
    durability: disposable  
    backup: false
```

Reset behavior

Provide two reset modes:

```
Repair app:
  preserve persistent data
  recreate runtime resources

Reset app data:
  delete persistent data after explicit confirmation
```

Uninstall behavior

Uninstall must offer:

```
Remove app only
Remove app and data
```

For true consumer simplicity, default should be:

```
Remove app only, preserve user data
```

But “Uninstall completely” must exist.

12. Health Checks

Health checks are mandatory for consumer packaging.

Without health checks, the platform cannot know when to show the app or recover from failures.

MVP health check

Support HTTP health checks:

```
health:
  ready:
    service: api
    path: /health
    timeoutSeconds: 120
```

The supervisor resolves the service/port route and polls until success.

Startup behavior

```
start db
start api
wait for api health
start web
wait for web route
start proxy
report Ready
```

Or, if Compose dependencies handle order:

```
start all services
poll required health checks
report Ready
```

Failure behavior

If health fails:

```
state: FailedRecoverable
user message: MyApp is taking longer than expected to start.
actions: Try Again, Repair, Show Details
```

13. Error Translation

Every infrastructure error needs a consumer-safe translation.

Examples

Internal:

```
no space left on device
```

User:

```
Your computer is low on storage. MyApp needs about 4.2 GB free.
```

Internal:

health check failed after 120s

User:

MyApp is taking longer than expected to start.

Internal:

WSL install requires reboot

User:

Windows needs to restart to finish setting up local app services.

Internal:

image import failed

User:

MyApp could not finish setup. The app package may be damaged.

Error object shape

```
{
  "code": "LOW_DISK_SPACE",
  "severity": "recoverable",
  "userTitle": "Your computer is low on storage",
  "userMessage": "MyApp needs about 4.2 GB free to finish setup.",
  "technicalMessage": "containerd import failed: no space left on device",
  "actions": ["RETRY", "EXPORT_DIAGNOSTICS"]
}
```

14. Diagnostics

Diagnostics should be available but hidden from the happy path.

User-facing diagnostics actions

```
Show Details
Export Diagnostics
Repair
Reset App Data
```

Diagnostic export should include

```
manifest.json
state.json
supervisor logs
service logs
runtime status
health check results
system info
free disk/memory summary
```

Do not include by default

```
secrets
raw environment variables
user data volumes
tokens
API keys
```

15. Repair

Repair is required for consumer-grade reliability.

Repair operation

```
stop app services
verify app bundle exists
verify runtime is reachable
recreate app network
recreate containers
preserve persistent volumes
restart services
run health checks
```

Repair should not

```
delete user data
change user settings
pull new images unless explicit update mode exists
```

16. Security Rules

Consumer Pack must be strict.

Defaults

```
bind only to 127.0.0.1
no privileged containers
no host network
no Docker socket mount
no arbitrary host filesystem mounts
no external networks
no external volumes
signed app bundle required eventually
health checks required
no mutable latest tags in locked bundle
```

Developer warnings

The CLI should fail loudly if the app cannot be safely packaged:

```
Consumer Pack cannot be built because this Compose file uses unsafe host access.
```

Signing

For 0→1 internal prototype, signing can be stubbed.

Before public distribution, signing is not optional:

```
Windows Authenticode signing
macOS Developer ID signing/notarization
internal app bundle signature
```

Unsigned installers will destroy trust and trigger scary OS warnings.

17. Consumer Certification

Add a `craterun certify` command early.

This should determine whether an app is safe and simple enough to package as a consumer installer.

Example output

```
Consumer readiness report
```

- ✓ app name configured
- ✓ icon configured
- ✓ frontend route configured
- ✓ health check configured
- ✓ all images build locally
- ✓ no privileged containers
- ✓ no host networking
- ✓ no Docker socket mounts
- ✓ no arbitrary host mounts
- ✓ all host ports auto-assigned
- ✓ persistent volumes declared
- ✓ repair path available

```
Certified: consumer-ready
```

Failed example

```
Consumer readiness failed
```

- x service api uses privileged: true
- x service worker mounts /var/run/docker.sock
- x service web maps fixed host port 3000
- x no health check configured

```
This app can be packaged as a Dev Pack, but not a Consumer Pack.
```

18. Installer/App State Machine

Implement this intentionally. Do not let this become scripts glued together.

States

NotInstalled
InstalledNotPrepared
CheckingSystem
PreparingRuntime
ImportingImages
CreatingVolumes
StartingServices
StartingProxy
WaitingForHealth
Ready
Stopping
Stopped
Repairing
FailedRecoverable
FailedBlocked
Uninstalling

State table

State	User Message	Primary Action	Failure State
CheckingSystem	Checking your computer...	automatic	FailedBlocked
PreparingRuntime	Preparing local app services...	automatic	FailedRecoverable
ImportingImages	Setting up MyApp...	automatic	FailedRecoverable
StartingServices	Starting MyApp...	automatic	FailedRecoverable
WaitingForHealth	MyApp is almost ready...	automatic	FailedRecoverable
Ready	MyApp is ready.	Open	n/a
Repairing	Repairing MyApp...	automatic	FailedRecoverable
FailedRecoverable	MyApp had trouble starting.	Repair	FailedBlocked
FailedBlocked	MyApp cannot start on this computer yet.	Show Details	n/a

Key distinction

FailedRecoverable means the app can retry/repair.

FailedBlocked means there is a system-level blocker, such as virtualization disabled, missing Windows feature, insufficient permissions, or unsupported OS.

19. 0→1 Build Milestones

Milestone 1 — Static App Bundle Compiler

Goal: given `docker-compose.yml` + `craterun.yml`, produce a structured app bundle.

Deliverables:

```
craterun init
craterun doctor
manifest parser
Compose parser
consumer subset validator
image build wrapper
OCI export
.appcrate directory output
```

Acceptance test:

A sample web+api app can be built into an appcrate folder containing manifest, locked compose plan, and image archives.

Milestone 2 — Local Supervisor Prototype

Goal: supervisor can read an app bundle and run services using a local developer runtime.

Use Docker or Podman on the developer machine for this milestone. Do not start with WSL complexity yet.

Deliverables:

```
supervisor binary
read manifest.json
import images into existing Docker/Podman
start services
start reverse proxy
run health check
return status API
```

Acceptance test:

craterun run-local launches the sample app and returns a local app URL.

Milestone 3 — Tauri Shell Prototype

Goal: native app shell talks to supervisor and displays the app.

Deliverables:

Tauri React shell
loading screen
status polling
webview navigation to app URL
basic error screen
Show Details panel

Acceptance test:

Launching the shell starts the supervisor, waits for readiness, and displays the sample web app.

Milestone 4 — Windows Managed Runtime Spike

Goal: prove a managed Windows runtime path.

Deliverables:

WSL runtime detection
managed distro creation/import spike
containerd startup inside distro
image import into managed runtime
container start/stop through adapter
logs/status retrieval

Acceptance test:

On a clean Windows 11 machine with required virtualization support, the supervisor prepares a private runtime and starts the sample app without Docker Desktop.

This is the riskiest milestone. Attack it early.

Milestone 5 — Native Installer

Goal: generate a Windows installer that installs the shell, supervisor, runtime assets, and app bundle.

Deliverables:

Windows installer generation
Start Menu shortcut
uninstall entry
post-install launch
per-user install path
basic clean uninstall

Acceptance test:

A non-technical tester can install MyApp from MyAppSetup.exe and open the app from the Start Menu.

Milestone 6 — Repair, Reset, Diagnostics

Goal: make failures survivable.

Deliverables:

Repair action
Reset app data action
Export diagnostics action
human-readable error mapping
service log capture
runtime status capture

Acceptance test:

If containers are manually broken, Repair restores the app without deleting persistent data.

Milestone 7 — Consumer Certification

Goal: prevent bad app packages from becoming consumer installers.

Deliverables:

```
craterun certify
unsafe Compose checks
required manifest checks
required health checks
required icon/name/version checks
consumer-pack build gate
```

Acceptance test:

```
Unsafe Compose files fail Consumer Pack builds with clear errors.
```

Milestone 8 — First External Test

Goal: validate the actual product promise.

Test with someone who is not a developer.

Acceptance criteria:

```
tester downloads installer
tester installs app without instructions
tester opens app
tester can close/reopen app
tester can uninstall app
tester never sees Docker/Compose/runtime terminology
tester never opens a terminal
tester does not need help
```

If this fails, do not add features. Fix the install experience.

20. Suggested Repo Structure

```
craterun/  
  crates/  
    craterun-cli/  
    craterun-supervisor/  
    craterun-core/  
    craterun-runtime/  
    craterun-bundle/  
  apps/  
    shell-tauri/  
  examples/  
    hello-web-api/  
    photobooth/  
  schemas/  
    craterun.schema.json  
    manifest.schema.json  
  docs/  
    architecture.md  
    consumer-pack.md  
    runtime-windows.md
```

Shared crates/modules

```
craterun-core:  
  shared types  
  app IDs  
  paths  
  state machine  
  errors  
  
craterun-bundle:  
  manifest parser  
  bundle reader/writer  
  appcrate format  
  
craterun-runtime:  
  runtime adapter traits  
  Docker/Podman dev adapter  
  WSL adapter later  
  
craterun-supervisor:  
  local control API  
  state machine execution
```

```
reverse proxy  
health checks
```

```
craterun-cli:  
  packaging commands  
  validator  
  image build/export orchestration
```

21. Technical Risks

Risk 1 — Windows managed runtime friction

This is the largest risk.

Potential blockers:

```
virtualization disabled  
WSL not installed  
Windows feature requires reboot  
enterprise policy disables WSL  
Defender flags runtime behavior  
filesystem performance issues  
port forwarding oddities
```

Mitigation:

```
spike early  
clear blocked-state messages  
fallback to existing Docker/Podman for dev mode  
document minimum Windows requirements
```

Risk 2 — Installer trust/security warnings

Unsigned installer warnings can ruin the consumer experience.

Mitigation:

```
for prototype, accept warnings
for public alpha, code sign
before broad release, invest in reputation/signing/notarization
```

Risk 3 — App size

Bundled images can be huge.

Mitigation:

```
zstd compression
image size report in CLI
base image recommendations
optional web installer later
layer dedupe later
```

CLI should show:

```
Bundle size report

web image: 187 MB
api image: 241 MB
db image: 146 MB
compressed bundle: 392 MB
```

Risk 4 — Compose compatibility expectations

Developers may expect arbitrary Compose apps to work.

Mitigation:

```
be explicit about Consumer Pack subset
provide Dev Pack separately
make errors clear
fail early
```

Risk 5 — Persistent data migrations

Updates are dangerous when persistent data exists.

Mitigation for MVP:

```
no automatic updates yet
manual reinstall only
preserve data by default
add migration hooks later
```

22. 0→1 Non-Goals

These should explicitly not be part of the first implementation:

```
marketplace
app store
background auto-update system
cross-platform parity
Kubernetes support
enterprise admin deployment
plugin ecosystem
cloud publishing
GPU workloads
multi-app shared runtime
developer account system
```

The first version should prove:

```
Can we turn one local Compose app into one Windows installer that a normal
person can install and run?
```

Nothing else matters until that is true.

23. Definition of Done for 0→1

The project reaches 0→1 when all of this is true:

A developer can package a supported Compose app with one command.
The output is a Windows installer.
A clean Windows 11 machine can install it without Docker Desktop.
The app opens in a native desktop window.
The user never sees Docker, Compose, WSL, containerd, ports, or terminal output.
The app can be closed and reopened.
The app can repair itself after a basic runtime/container failure.
The app can be uninstalled cleanly.
The app preserves or removes user data according to uninstall choice.

The final test should be brutal:

Hand the installer to a non-technical person and say nothing.

If they can install and run the app, the 0→1 worked.

If they need instructions, the product is not ready.

24. Implementation Priority

Recommended order:

1. Manifest schema
2. Compose subset validator
3. App bundle compiler
4. Existing-runtime supervisor
5. Reverse proxy
6. Tauri shell
7. Windows WSL runtime adapter spike
8. Installer generation
9. Repair/reset/diagnostics
10. Consumer certification gate

Do not start with the perfect runtime abstraction. Start with the vertical slice, but keep adapter boundaries clean enough that Docker Desktop does not become a permanent dependency.

25. Key Product Principles

1. **The user installs an app, not a runtime.**
2. **Docker is an internal implementation detail.**
3. **The developer machine does the heavy lifting.**
4. **Consumer Pack must be strict and safe.**

5. **Every raw infrastructure error must be translated.**
 6. **Repair must preserve user data.**
 7. **Uninstall must be clean.**
 8. **No setup questions unless absolutely unavoidable.**
 9. **Health checks are mandatory.**
 10. **The first version should be narrow and excellent, not broad and cursed.**
-

26. Open Questions for Architect

These should be answered during the first technical spike:

1. What is the most reliable Windows managed runtime approach?
 2. WSL2 imported distro + containerd?
 3. Podman machine?
 4. bundled lightweight VM alternative?
 5. Can the managed runtime be installed per-user without admin privileges?
 6. What Windows features are required, and when is reboot unavoidable?
 7. Can image import/startup time be kept acceptable for a consumer app?
 8. Should the supervisor be a background process, Windows service, or child process of the shell for MVP?
 9. How should the shell securely communicate with the supervisor?
 10. named pipe?
 11. localhost HTTP with random token?
 12. What is the smallest supported Compose subset that still covers useful real apps?
 13. How cleanly can uninstall remove WSL/runtime artifacts?
 14. What signing requirements are needed before public testing?
 15. How should we represent app data backups in the manifest?
-

27. Recommended First Prototype App

Build the first test around a simple but realistic app:

```
React frontend  
.NET or Node API  
Postgres database  
one persistent volume  
/api route  
/health endpoint
```

This is enough to test:

```
frontend/backend routing  
container startup order  
database persistence  
health checks  
volume handling  
reverse proxy  
native app shell  
repair behavior  
uninstall behavior
```

Avoid a toy single-container app. It will not expose enough of the real complexity.

28. Final Guidance

The fastest way to ruin this project is to make it a general-purpose Docker Compose GUI.

The correct wedge is narrower and stronger:

A compiler from safe, supported Compose apps into consumer-grade desktop installers.

The consumer path must be ruthless:

```
Download. Install. Open.
```

Everything else belongs behind the curtain.